

DEEP TECHNICAL GUIDE

AI IN CLOUD: INTERNALS, ARCHITECTURE & IMPLEMENTATION

Amazon Bedrock · SageMaker · Vertex AI · Microsoft Foundry · Copilot Studio

Design · Architecture · Data Flow · Implementation · Failure Modes · Trade-offs

Version 3.0 — May 2026 — Principal Training Architect Series

Table of Contents

Chapter 1: Inference Engine Internals — How Cloud AI Actually Serves Models

1.1 The Problem Inference Engines Solve

Serving a large language model in production is fundamentally different from running it in a notebook. A production inference engine must solve simultaneous challenges: multi-tenancy (thousands of customers sharing the same GPU fleet), latency SLA enforcement (P99 under 200ms for first token), throughput maximization (tokens per second per dollar), fault tolerance (GPU failures are routine at scale), and security isolation (no customer can observe another's prompts or completions). Each cloud provider has built a proprietary inference engine to solve these problems, and understanding their design is essential for making informed architectural decisions.

1.2 AWS Project Mantle — Zero-Operator-Access Distributed Inference

1.2.1 Architecture & Design Philosophy

Project Mantle is Amazon Bedrock's next-generation distributed inference engine, purpose-built for large-scale model serving. It was publicly disclosed in December 2025 by Anthony Liguori, AWS VP and Distinguished Engineer for Amazon Bedrock. Mantle represents a ground-up reimagining of Bedrock's serving layer, with security as the top design priority.

Mantle's architecture follows the principles of the AWS Nitro System: **zero operator access (ZOA)**. This means there is no technical mechanism for any AWS operator to sign in to underlying compute systems or access any customer data, including inference prompts or completions. The design intentionally excludes interactive communication tools like SSH, AWS Systems Manager Session Manager, and serial consoles. All inference software updates must be signed and verified before deployment.

Key architectural components of Mantle:

- **EC2 Instance Attestation:** Mantle uses cryptographically signed attestation measurements from the Nitro Trusted Platform Module (NitroTPM) to configure a hardened, constrained, and immutable compute environment. The services responsible for handling model weights and conducting inference are backed by NitroTPM attestation.
- **End-to-End Encryption:** When a customer calls a Mantle endpoint (bedrock-mantle.[region].api.aws), prompts leave the customer's environment through TLS and remain encrypted all the way to the Mantle service. No operator—from AWS, the customer, or a model provider—can access the data at any point.
- **Model Provider Isolation:** Inferencing occurs only within Amazon Bedrock-owned accounts. Model providers have no mechanism to access customer data because they have no access to the inference accounts.
- **Automated Operations:** Systems are administered through automation and secure APIs, not human operators. This eliminates the entire class of insider threat vectors that plague traditional infrastructure.

1.2.2 Mantle's Dual-Endpoint Architecture

Bedrock exposes two distinct endpoint families, each powered by different serving infrastructure:

Endpoint	API Compatibility	Powered By	Model Scope	Use Case
bedrock-runtime	AWS Converse API, InvokeModel, Chat Completions	Legacy Bedrock serving	Proprietary models (Claude, Nova, Titan)	Existing Bedrock applications
bedrock-mantle	OpenAI-compatible (Responses API, Chat Completions)	Project Mantle (ZOA)	Open-weight models (DeepSeek, Gemma, Mistral, GPT OSS, Nemotron, OpenAI)	New applications, OpenAI migrations

⚠ **Critical implementation detail:** AWS PrivateLink is now supported for both endpoints. For *bedrock-mantle*, PrivateLink was added in February 2026 across 14 AWS regions. Always use PrivateLink in production to ensure inference traffic never traverses the public internet.

1.2.3 Capacity Management & Quality of Service

Mantle provides several QoS mechanisms that are invisible to the developer but critical for production reliability:

- **Automated Capacity Management:** Mantle dynamically allocates GPU resources across its fleet based on real-time demand. Unlike the legacy bedrock-runtime endpoint where customers hit rate limits during traffic spikes, Mantle uses unified pools that provide higher default quotas.
- **Cross-Region Inference (CRIS):** For workloads with data sovereignty requirements, CRIS routes inference within a configured geographic boundary. For example, Australian workloads stay within Sydney and Melbourne regions. This is not load balancing—it is geographic containment with failover.
- **Projects API:** Announced March 2026, Projects provide logical isolation within a single Bedrock account. Each project gets independent IAM-based access control and cost attribution tags. This enables multi-team and multi-environment isolation without separate AWS accounts.

1.2.4 Failure Modes & Mitigation

Failure Mode	Symptoms	Root Cause	Mitigation Strategy
ThrottlingException	429 status, RetryAfter header	Request rate exceeds provisioned/on-demand quota	Implement exponential backoff with jitter; use Provisioned Throughput for steady workloads; leverage CRIS for geographic distribution
ModelTimeoutException	408 status, incomplete response	Complex prompt exceeds inference timeout; GPU memory pressure	Reduce prompt length; split into multiple calls; use smaller model for pre-processing; request quota increase
ValidationException	400 status, specific error message	Malformed request, unsupported parameters, token limit exceeded	Validate input before sending; check model-specific parameter limits; use tokenizer to pre-count tokens
ServiceUnavailableError	503 status	Fleet-level capacity exhaustion; regional outage	Multi-region deployment; circuit breaker pattern; fallback to alternative model; queue and retry
InternalServerError	500 status, no details	GPU failure, inference crash, model loading error	Retry with exponential backoff; Mantle auto-heals by rerouting to healthy instances; monitor CloudTrail for patterns
Model Deprecation	Model returns error after retirement date	Model version reached end-of-life	Subscribe to model lifecycle notifications; maintain model version inventory; test successor models before migration deadline

1.2.5 Intelligent Prompt Routing — Internal Architecture

Bedrock’s Intelligent Prompt Routing is a cost-optimization layer that sits between the API gateway and model fleet. It classifies incoming requests by complexity and routes them to the most cost-effective model that can produce acceptable quality:

- **Classification Stage:** A lightweight classifier (running on CPU, not GPU) analyzes the prompt structure, token count, required capabilities (tool use, vision, reasoning), and historical quality scores.
- **Routing Decision:** Simple queries (FAQ, classification, extraction) are routed to lightweight models (Nova Micro, Ministral 3B). Complex queries (multi-step reasoning, code generation, analysis) are routed to frontier models (Claude Opus, GPT-5).
- **Quality Feedback Loop:** Response quality metrics are fed back to the classifier to continuously improve routing accuracy. AWS reports up to 30% inference cost reduction with no quality degradation.

⚠ **Trade-off:** Intelligent Prompt Routing adds 10–50ms of latency for the classification step. For ultra-low-latency applications (real-time voice, gaming), bypass routing and specify the model directly.

1.3 Google Cloud Inference Infrastructure — TPUs, Pathways & vLLM

1.3.1 TPU Architecture for AI Inference

Google’s inference infrastructure is unique in the industry because it runs primarily on custom Tensor Processing Units (TPUs) rather than NVIDIA GPUs. As of Cloud Next 2026, Google operates two specialized TPU families:

Specification	TPU 8t (Training)	TPU 8i (Inference)
Purpose	High-throughput training	Low-latency inference and RL
Host CPU	Custom Arm-based Axion (replaces x86)	Custom Arm-based Axion (replaces x86)
Superpod Scale	9,600 chips, 2 PB shared HBM	1,152 chips per pod (direct-connect topology)
Compute per Pod	121 ExaFLOPS	Not disclosed (optimized for latency, not raw FLOPS)
Key Innovation	10x faster storage access; near-linear scaling to 1M chips via Virgo Network + JAX + Pathways	3x more on-chip SRAM for larger KV caches on-silicon; eliminates off-chip memory latency for attention
Performance vs. Previous Gen	~2.7x better training perf-per-dollar	~80% better inference perf-per-dollar
Network	Virgo Network (new data center fabric)	Direct-connect ICI topology (no switch hops)
Software Stack	JAX + Pathways + XLA	Optimized vLLM + Pathways for serving

1.3.2 Pathways — Google’s Distributed Orchestration System

Pathways is Google’s internal system for orchestrating computation across thousands of TPUs. For inference, Pathways handles:

- **Model Sharding:** Automatically partitions large models across multiple TPU chips using tensor parallelism (splitting individual operations) and pipeline parallelism (splitting model layers across chips).

- **KV Cache Management:** The key-value cache is the primary memory bottleneck in autoregressive generation. TPU 8i's 3x SRAM increase means larger KV caches fit on-chip, eliminating the HBM roundtrip that dominates latency in long-context inference.
- **Continuous Batching:** Groups incoming requests into efficient batches without waiting for fixed batch windows. New requests are injected into running batches as previous requests complete tokens.
- **Speculative Decoding:** Uses a small draft model to generate candidate tokens in parallel, then verifies them with the full model. This amortizes the cost of the full model's forward pass across multiple tokens.

1.3.3 GKE for AI Inference — Cold Start & Scale-Out

At Cloud Next 2026, Google announced that GKE now delivers dramatically faster cold starts and scale-out for AI inference workloads, plus agent sandboxes capable of deploying 300 sandboxes per second per cluster with sub-second time to first instruction. The implementation details:

- **Pre-warmed Node Pools:** GKE maintains warm pools of TPU/GPU nodes with pre-loaded model weights. When autoscaler triggers, new replicas start serving in seconds rather than minutes.
- **Model Caching on Managed Lustre:** The new Managed Lustre storage system provides 10 TB/s throughput. Model weights are cached on Lustre, eliminating the S3/GCS download step that traditionally dominated cold start time.
- **Agent Sandboxes:** Each agent session runs in a lightweight, isolated sandbox. The 300/second deployment rate is achieved through gVisor (Google's container sandbox) with pre-initialized runtimes.

1.3.4 Failure Modes Specific to Vertex AI / Agent Platform

Failure Mode	Symptoms	Root Cause	Mitigation
Quota Exhaustion	RESOURCE_EXHAUSTED error	GSU (Generative Scale Unit) limit reached	Purchase additional Provisioned Throughput; use global endpoint for overflow; implement client-side queuing
Model Retirement	INVALID_ARGUMENT with model version error	Model version deprecated (e.g., gemini-2.0-flash-001 blocked for new projects from March 2026)	Track model lifecycle dates; use stable model aliases; test successor models quarterly
Context Window Overflow	INVALID_ARGUMENT: max tokens exceeded	Input + output exceeds model's context limit	Pre-count tokens with countTokens API; implement chunked processing; use summarization for long documents
Grounding Failures	Low grounding scores, hallucinated content	RAG retrieval returned irrelevant chunks; embedding model mismatch	Tune chunk size and overlap; use Gemini Embedding 2 for multimodal retrieval; validate retrieval quality with evaluation service
Agent Loop Stuck	Agent repeatedly calls same tool, burns tokens	Poor tool description, ambiguous instructions, missing exit condition	Define explicit exit criteria; set max_iterations; add timeout; improve tool descriptions with examples
TPU Preemption	Request fails mid-stream	Spot/preemptible TPU reclaimed	Use on-demand TPUs for production; implement checkpointing; use global endpoint for automatic failover

1.4 Microsoft Azure Inference Architecture

1.4.1 Foundry's Layered Inference Stack

Microsoft Foundry's inference architecture operates as a layered stack that separates concerns between the API gateway, model serving, and security enforcement:

- **Layer 1 — Azure Front Door / API Management:** Global load balancing, rate limiting, authentication (Entra ID / API keys), and request routing. This layer determines which model deployment receives the request based on deployment name, region, and capacity.
- **Layer 2 — AI Gateway:** A new component in the 2026 architecture that provides centralized policy enforcement. The AI Gateway intercepts every request and applies guardrails, content safety checks, token budget enforcement, and audit logging before forwarding to the model backend.
- **Layer 3 — Model Serving Backend:** The actual inference infrastructure. For OpenAI models, this runs on Azure's custom inference clusters (co-managed with OpenAI). For open-source models, this uses Azure's GPU fleet with optimized serving frameworks (vLLM, TensorRT-LLM). For partner models (Anthropic, Mistral), this operates in dedicated, isolated partitions.
- **Layer 4 — Guardrails Post-Processing:** After the model generates a response, the AI Gateway applies output-side safety checks, PII detection, and task adherence verification before returning the response to the customer.

1.4.2 Deployment Types — Technical Implementation

Deployment Type	Infrastructure	Billing	Latency Profile	Data Residency	Best For
Global Standard	Shared GPU fleet, routed globally	Pay-per-token	Variable (50–500ms TTFT)	No geographic guarantees	Dev/test, variable workloads, cost-sensitive
Data Zone Standard	Shared fleet within geographic zone	Pay-per-token (slight premium)	Variable within zone	Within zone (US, EU, etc.)	Regulated industries, GDPR compliance
Global Provisioned	Dedicated capacity, routed globally	Hourly per PTU	Consistent (lower P99)	No geographic guarantees	Production with SLA requirements
Hosted Agents (Preview)	Managed compute (North Central US only)	Runtime + model tokens	~100–300ms per agent step	North Central US	Rapid agent deployment, no infra management
Bring-Your-Own-Runtime	Customer's AKS/App Service/Container Apps	Customer infra + Foundry API tokens	Customer-controlled	Customer-controlled	Maximum control, multi-region, custom networking

⚠ Critical limitation: Hosted agents are available only in North Central US as of March 2026. For UK/EU workloads with data residency requirements, use the Bring-Your-Own-Runtime pattern with Foundry endpoints in closer regions. Azure is expanding regional availability but has not announced dates.

1.4.3 Foundry Agent Service — Internal Data Flow

The Foundry Agent Service data flow is critical to understand for security and compliance analysis. Here is the step-by-step flow for a single agent invocation:

- **Step 1 — Authentication:** Request arrives at Foundry endpoint. Entra ID validates the caller's identity and checks RBAC permissions. For MCP tools with OAuth Identity Passthrough, the user's delegated token is attached to the request context.

- **Step 2 — Pre-Processing Guardrails:** The AI Gateway inspects the input. Content Safety checks for harmful content. PII NextGen scans for personal data. Task Adherence validates the request is within the agent's defined scope. If any check fails, the request is blocked and an audit event is logged.
- **Step 3 — Agent Reasoning Loop:** The request enters the Responses API runtime. The model (specified in the agent definition) receives the system prompt, conversation history, tool definitions, and user message. The model generates a response that may include tool calls.
- **Step 4 — Tool Execution:** If the model requests a tool call, the runtime executes it. For MCP tools, the request is forwarded to the MCP server (either Foundry-hosted at mcp.ai.azure.com or customer-hosted). The tool response is fed back to the model. For tools with private networking, traffic stays within the customer's VNet.
- **Step 5 — Post-Processing Guardrails:** The model's final response passes through output-side guardrails. Tool invocation guardrails check what tools were called and validate their responses. Palo Alto Prisma AIRS and Zenity integrations detect prompt injection, toxic content, malicious URLs, and sensitive data leakage.
- **Step 6 — Tracing & Observability:** OpenTelemetry traces capture the full execution path: model latency, tool invocation duration, guardrail evaluation time, and token counts. Traces flow to Azure Monitor. Continuous evaluation samples production traffic and runs built-in evaluators for quality, safety, and performance.
- **Step 7 — Memory Consolidation:** If memory is enabled, the runtime extracts preferences, facts, and key context from the conversation turn, consolidates them via LLM (merging duplicates, resolving conflicts), and stores them for retrieval in future sessions.

Chapter 2: RAG Pipeline Deep-Dive — Data Ingestion, Embedding & Retrieval

2.1 The RAG Data Pipeline — End-to-End Architecture

Retrieval-Augmented Generation is the dominant pattern for grounding AI responses in enterprise data. All three clouds implement RAG as a multi-stage pipeline with distinct design choices at each stage. Understanding these internals is critical for diagnosing quality issues, optimizing cost, and meeting latency SLAs.

2.1.1 Stage 1 — Ingestion & Parsing

Raw documents (PDFs, DOCXs, HTML, images, audio, video) must be converted to text before embedding. Each platform handles parsing differently:

Platform	Parser Options	Multimodal Support	Custom Parsing
AWS Bedrock KB	Built-in parser; Amazon Bedrock Data Automation (BDA) for complex documents; Amazon Textract for forms/tables	Nova multimodal embeddings for audio (1–30s chunks, default 5s), video (chunk duration configurable), images	Custom Lambda function for pre-processing; custom chunking via Lambda transformation
Google Vertex AI	Document AI for OCR and layout detection; Gemini for visual parsing	Gemini Embedding 2 (natively multimodal: text, image, video, audio, documents in single embedding space)	Cloud Functions for custom parsers; Document AI custom processors
Azure Foundry	Azure AI Document Intelligence for PDFs/forms; Foundry IQ for enterprise content	Azure AI Search multimodal embeddings	Custom skills pipeline in Azure AI Search; Azure Functions for pre-processing

2.1.2 Stage 2 — Chunking Strategies

Chunking is the most impactful parameter in RAG quality. The wrong chunk size produces either too-narrow context (missing information) or too-broad context (diluted relevance). Here is a technical analysis of each strategy:

- **Fixed-Size Chunking:** Splits text at a fixed token count with configurable overlap. Simplest to implement and debug. Best for homogeneous document types (e.g., all similar-length FAQ entries). Failure mode: splits mid-sentence, breaking semantic meaning. Overlap mitigates this but increases storage and embedding cost.
- **Semantic Chunking (Bedrock):** Creates embeddings for individual sentences, then groups semantically similar sentences into chunks based on a similarity threshold (50–99th percentile). The breakpoint threshold controls how different adjacent groups must be to trigger a split. Recommended threshold: 95. Max token size: 20–8,192 (recommended 300). Best for documents with topic shifts. Failure mode: sensitive to embedding model quality; poor embeddings produce poor boundaries.
- **Hierarchical Chunking (Bedrock):** Creates parent-child chunk relationships. Small child chunks are used for precise retrieval; when a child is retrieved, it is replaced with its parent chunk during augmentation. This balances embedding precision with context completeness. Configuration: parent max tokens (e.g., 1500) and child max tokens (e.g., 300) with overlap between consecutive parents and children. Failure mode: returned result count may be less than requested because multiple children map to the same parent.

- **Contextual Chunking (Vertex AI):** Uses surrounding sentence buffer to contextualize each chunk's embedding. Buffer size determines how many adjacent sentences are included in the embedding calculation (buffer=1 means current + previous + next sentence). Best for narrative content. Failure mode: large buffer introduces noise.

⚠ Production recommendation: Start with fixed-size chunking (256–512 tokens, 10–20% overlap) for initial deployment. Measure retrieval quality with your actual queries. If quality is insufficient, switch to semantic chunking. Hierarchical chunking is valuable when documents have clear structural hierarchy (headings, sections) and queries range from specific facts to broad summaries.

2.1.3 Stage 3 — Embedding Models

The embedding model translates text chunks into vector representations. Model choice directly impacts retrieval quality, storage cost, and query latency:

Model	Provider	Dimensions	Context Window	Modalities	Key Feature
Titan Text Embeddings V2	AWS	256 / 512 / 1024	8,192 tokens	Text	Configurable dimensions for cost/quality trade-off
Nova Multimodal Embeddings	AWS	1024	Text: 512 tokens; Audio/Video: 1–30s chunks	Text, Image, Audio, Video	First multimodal embedding on Bedrock
Gemini Embedding 2	Google	768 / 3072	8,192 tokens	Text, Image, Video, Audio, Documents	Natively multimodal; single embedding space for all modalities
text-embedding-3-large	OpenAI (via Azure)	256–3072	8,191 tokens	Text	Matryoshka encoding: truncate dimensions without retraining
Cohere Embed v4	Cohere (all clouds)	1024	512 tokens	Text	Optimized for search, classification, and clustering

2.1.4 Stage 4 — Vector Stores

The vector store is where embedded chunks are indexed for similarity search. Each platform supports multiple backends with distinct performance characteristics:

Vector Store	Platform	Index Algorithm	Hybrid Search	Scaling Model	Best For
OpenSearch Serverless	AWS	HNSW (Faiss engine)	Yes (k-NN + BM25)	Serverless (auto-scaling OCUs)	General RAG with filtering; managed, no capacity planning
Aurora PostgreSQL (pgvector)	AWS	IVFFlat / HNSW	Yes (pgvector + tsvector)	Provisioned (instance-based)	Applications already on Aurora; transactional + vector in one DB
S3 Vectors (Preview)	AWS	Proprietary	No	Serverless	Cost-optimized storage for large, infrequently queried corpora
Neptune Analytics	AWS	Graph + Vector	Yes (GraphRAG)	Provisioned	Knowledge graphs; relationship-rich domains (pharma, legal)
Vector Search 2.0	Google	HNSW + ScaNN	Yes (vector + full-text + semantic re-ranking)	Serverless (auto-scaling)	Advanced retrieval with built-in re-ranking; private PSC/PGA support

Vector Store	Platform	Index Algorithm	Hybrid Search	Scaling Model	Best For
AlloyDB for PostgreSQL	Google	IVFFlat / HNSW (pgvector)	Yes	Provisioned	PostgreSQL workloads with vector search; high-throughput OLTP + vector
Azure AI Search	Azure	HNSW	Yes (vector + keyword + semantic re-ranking)	Provisioned (search units)	Enterprise search with facets, filters, and security trimming
Cosmos DB (vector)	Azure	DiskANN	Yes	Serverless or provisioned	Global distribution; multi-model (document + vector); low-latency reads

2.1.5 Stage 5 — Retrieval & Augmentation

At query time, the retrieval pipeline converts the user's question into an embedding, searches the vector store, and augments the prompt with retrieved chunks:

- **Query Reformulation (Bedrock):** Before embedding, Bedrock can rewrite the user's query to improve retrieval quality. Multi-query reformulation generates multiple paraphrased queries and merges results. This helps when user questions are ambiguous or use different vocabulary than the source documents.
- **Hybrid Search:** Combines vector similarity (semantic matching) with keyword/BM25 search (exact term matching). Critical for domains with specific terminology (legal, medical, financial) where exact terms matter as much as semantic meaning.
- **Re-Ranking:** A dedicated model scores the relevance of retrieved chunks to the original query and re-orders them. Cohere Rerank v4.0 (available on all three clouds) and Azure AI Search's built-in semantic ranker are the most common options. Re-ranking significantly improves precision at a small latency cost (20–50ms).
- **Metadata Filtering:** Pre-filters the vector search by metadata (document type, date range, department, access level). Reduces the search space and improves relevance. All platforms support metadata filtering, but implementation varies. Azure AI Search supports security trimming (filtering based on the user's identity/group membership).
- **Cross-Corpus Retrieval (Vertex AI, Preview):** Retrieve from multiple RAG corpora simultaneously. Useful when knowledge is distributed across separate indexes (e.g., product docs + support tickets + internal wiki).

2.1.6 RAG Failure Modes & Debugging

Failure Mode	Symptoms	Diagnosis	Fix
Low Recall	Model says "I don't have information" despite data existing in corpus	Query embedding doesn't match chunk embeddings; chunks too large; vocabulary mismatch	Add query reformulation; reduce chunk size; try semantic chunking; add keyword/hybrid search
Low Precision	Model includes irrelevant information in response	Chunks too broad; insufficient re-ranking; metadata not filtered	Reduce chunk size; add re-ranking step; implement metadata filtering; increase similarity threshold
Hallucination Despite RAG	Model generates plausible but incorrect facts	Retrieved chunks don't contain the answer; model ignores context; context window overflow	Add grounding check (Bedrock Guardrails / Model Armor); reduce temperature; instruct model to cite sources; verify chunks contain the answer

Failure Mode	Symptoms	Diagnosis	Fix
Stale Data	Model returns outdated information	Ingestion pipeline not syncing; vector store not updated	Set up incremental sync schedule; use change data capture; monitor ingestion job success/failure
Embedding Drift	Retrieval quality degrades over time	Embedding model updated/changed; new data has different distribution	Re-embed entire corpus when changing embedding model; monitor retrieval metrics; A/B test embedding models
Context Window Overflow	Truncated context, missing critical chunks	Too many chunks retrieved; chunks too large; system prompt too long	Limit retrieval to top-k (3–5 chunks); use hierarchical chunking (parent replacement); compress system prompt

Chapter 3: Agent Architecture — Orchestration, Tools & Multi-Agent Systems

3.1 The Agent Execution Loop

All cloud AI agent frameworks implement a variation of the ReAct (Reasoning + Acting) pattern: the model receives a task, reasons about what to do, selects and invokes tools, observes the results, and iterates until the task is complete or a termination condition is met. The architectural differences lie in how each platform implements isolation, tool connectivity, memory, and governance.

3.1.1 AWS Bedrock AgentCore — MicroVM-Per-Session

AgentCore's managed harness (preview, April 2026) provides the most isolated agent execution environment among the three clouds:

- **Execution Model:** Each agent session gets its own microVM with dedicated filesystem and shell access. This eliminates cross-session data leakage and provides true process isolation.
- **Model Agnosticism:** The harness supports switching models mid-session. Configuration set at create time can be overridden per invocation, enabling A/B testing without redeployment.
- **Export Path:** When the managed harness's constraints are too limiting, developers export to Strands SDK (open-source) for full code control. The exported code is compatible with any compute target (Lambda, ECS, EKS).
- **Policy Engine:** Cedar policies (AWS's declarative authorization language) define what actions an agent can take. Policies are verified outside the agent's reasoning loop before reaching tools or data. This means a compromised model cannot bypass authorization.
- **Memory:** Episodic memory allows agents to learn from past interactions. Memory is stored per-agent, not shared across agents. The memory system is read-only during inference—agents observe memories but cannot modify them during execution.

3.1.2 Google ADK — Graph-Based Multi-Agent Orchestration

Google's Agent Development Kit (ADK, open-source, preview since Cloud Next 2025) takes a fundamentally different approach:

- **Graph Architecture:** Agents are defined as nodes in a directed graph. Edges represent delegation relationships. A parent agent can delegate sub-tasks to child agents, which can further delegate. This creates a tree of specialized agents collaborating on a complex task.
- **Model Agnosticism:** ADK supports any model available in Model Garden (Gemini, Claude, Llama, DeepSeek). Models are specified per-agent, so a planning agent can use Gemini 3.1 Pro while an execution agent uses Gemini 2.5 Flash for cost efficiency.
- **Agent Engine Runtime:** The managed, serverless runtime handles compute provisioning and scaling. Billing is per vCPU-hour for the runtime plus per-token for model calls.
- **Sessions & Memory Bank (GA):** Sessions persist conversation state across turns. Memory Bank extracts, consolidates, and retrieves long-term memories across sessions. Memory entries cost \$0.25 per 1,000 events.
- **Agent Garden (Preview):** A marketplace of pre-built agents that can be composed into workflows. Reduces development time for common patterns (customer support, data analysis, document processing).

- **Agent Simulation:** A dedicated environment for stress-testing agents before production deployment. Simulates user interactions, edge cases, and adversarial inputs.

3.1.3 Microsoft Foundry Agent Service — Responses API Runtime

Foundry’s agent architecture is built directly on the OpenAI Responses API, making it wire-compatible with the OpenAI agent ecosystem:

- **Three Agent Types:** Prompt agents (declarative, GA): defined by system prompt and tool bindings, no custom code. Hosted agents (preview): custom code running in Foundry-managed compute. Workflow agents (preview): multi-step orchestration with conditional logic and human-in-the-loop.
- **Identity Model:** Unpublished agents run with the project’s system-assigned managed identity. Published agents get a distinct identity provisioned by Foundry. This distinction is critical: publishing requires reconfiguring permissions because project-level permissions don’t transfer.
- **MCP Tool Connectivity:** Full spectrum support: Foundry-hosted MCP (mcp.ai.azure.com), customer-hosted MCP, and OAuth Identity Passthrough (agent acts on behalf of the user, not as a system identity).
- **Multi-Agent (via Microsoft Agent Framework):** The open-source framework (1.0.0rc1, February 2026) supports Python and .NET. It provides durable execution (agents persist and recover across restarts), Microsoft Purview integration (classifying and securing prompts, outputs, and responses), and compatibility with AG-UI and ChatKit protocols.
- **Continuous Evaluation:** Samples production traffic and runs 13 built-in evaluators (quality, safety, availability, performance). Results surface through Azure Monitor dashboards with alerting on regressions.

3.2 Tool Connectivity Protocols — MCP & A2A

3.2.1 Model Context Protocol (MCP)

MCP is the emerging standard for connecting AI agents to external tools and data sources. All three clouds now support MCP, but implementation maturity varies:

Feature	AWS (AgentCore)	Google Cloud (Agent Platform)	Azure (Foundry)
MCP Server Hosting	Customer-hosted; stateful MCP server support	Managed remote MCP servers for databases; Cloud API Registry	Cloud-hosted (mcp.ai.azure.com); customer-hosted
Authentication	IAM-based; per-tool permissions	IAM + OAuth	Entra ID; OAuth Identity Passthrough; per-tool authentication
Stateful Sessions	Supported (March 2026)	Supported via Sessions API	Supported via conversation management
Available Connectors	Custom via Lambda/API Gateway	70+ managed database MCP servers; custom	1,400+ enterprise connectors (Logic Apps); custom
Network Isolation	VPC endpoints; PrivateLink	VPC-SC; Private Google Access	Private VNet; no public egress for Standard Setup

3.2.2 Agent-to-Agent (A2A) Protocol

A2A allows agents to invoke other agents across organizational and platform boundaries. This is critical for multi-vendor agent ecosystems:

- **Azure Foundry A2A (Preview, December 2025):** Agents call any A2A-protocol endpoint with explicit auth and clean call/response semantics. This is the structured evolution of Connected Agents. The A2A tool surfaces in the Foundry portal's Tools tab alongside MCP servers.
- **Google A2A (Preview, Cloud Next 2025):** Part of the ADK graph model. Agents can delegate to external A2A endpoints as child agents. The protocol includes discovery (what can this agent do?), invocation (do this task), and status (is it done?).
- **AWS:** No dedicated A2A protocol as of May 2026. Multi-agent patterns are implemented via AgentCore's managed harness or via Strands SDK with custom orchestration logic.

Chapter 4: Security Architecture — Defense-in-Depth for Cloud AI

4.1 Data Flow Security Analysis

Every AI request creates a data flow that must be secured at each hop. The following analysis traces a typical request through each cloud’s security stack:

4.1.1 AWS Bedrock — Security Data Flow

- **Hop 1 (Client → API Gateway):** TLS 1.2+ termination at AWS API Gateway. SigV4 request signing validates caller identity. IAM policy evaluation determines authorization. If PrivateLink is configured, traffic never leaves the AWS backbone.
- **Hop 2 (API Gateway → Guardrails):** Request passes through configured guardrails. Content filtering, PII detection (configurable entity types + custom regex), denied topic enforcement, and word filtering are evaluated in sequence. Failed checks return a descriptive error and log to CloudTrail.
- **Hop 3 (Guardrails → Inference):** For bedrock-mantle (open-weight models): request enters the ZOA Mantle environment. NitroTPM attestation validates compute integrity. No operator access exists. For bedrock-runtime (Claude, Nova): request enters the legacy serving environment with AWS-standard operational security.
- **Hop 4 (Inference → Response):** Model response passes through output-side guardrails. Grounding check validates response against retrieved context. PII re-detection on output. Response returned to client via TLS.
- **Hop 5 (Audit):** CloudTrail records the API call metadata. If model invocation logging is enabled (opt-in), the full prompt and completion are logged to S3 or CloudWatch Logs with KMS encryption.

4.1.2 Guardrails & Safety Systems — Technical Comparison

Capability	AWS Guardrails	Google Model Armor	Azure Content Safety + PII NextGen
Content Filtering	Configurable severity thresholds (NONE/LOW/MEDIUM/HIGH) for hate, insults, sexual, violence	Safety filter settings (BLOCK_NONE through BLOCK_MOST_INCLUSIVE) for harassment, hate, sexually explicit, dangerous	Configurable severity (safe/low/medium/high) for violence, hate, sexual, self-harm + custom categories
PII Detection	Built-in entities (SSN, email, phone, etc.) + custom regex patterns; action: BLOCK or ANONYMIZE	Not built into Model Armor; available via DLP API integration	PII NextGen: conversational and document PII detection with preview features; configurable redaction
Prompt Injection Detection	Contextual grounding check on input	Prompt injection detection at runtime	Third-party: Palo Alto Prisma AIRS, Zenity; native prompt injection detection in guardrails
Tool Safety	Cedar policies verify actions outside reasoning loop	Agent identity IAM; Model Armor on tool outputs	Guardrail intervention points for tool invocations and tool responses (GA March 2026)
Task Adherence	Not built-in (implement in system prompt)	Not built-in	Native guardrail risk type: block or annotate off-task tool calls (March 2026)

Capability	AWS Guardrails	Google Model Armor	Azure Content Safety + PII NextGen
Grounding Validation	Contextual grounding check compares response to retrieved context	Grounding with enterprise data or Google Search	Foundry IQ citation-backed answers with Purview-based security
Third-Party Integration	Custom Lambda for additional checks	Partner guardrails via Marketplace	Prisma AIRS, Zenity (GA March 2026); more via Azure Marketplace

4.2 Encryption Architecture

Encryption for cloud AI spans three domains: data at rest (stored prompts, embeddings, model weights), data in transit (API calls, inter-service communication), and data in use (during inference computation). Here is the detailed analysis:

- **Data at Rest:** All platforms encrypt stored data by default. AWS uses AES-256 via KMS with optional customer-managed keys (CMK). Google uses AES-256 via Cloud KMS with optional CMEK. Azure uses AES-256 via Key Vault with optional CMK (service-level CMK for AI Search added March 2026). Fine-tuning data, vector store contents, and agent memory are all encrypted.
- **Data in Transit:** TLS 1.2+ is mandatory on all platforms. AWS PrivateLink, Google Private Service Connect, and Azure Private Link ensure traffic stays on the provider's backbone. For Mantle specifically, encryption is end-to-end from client to ZOA compute—there is no point where unencrypted data exists outside the Nitro enclave.
- **Data in Use (Confidential Computing):** This is the frontier. AWS Mantle's NitroTPM attestation provides the strongest guarantee: no operator can access data during computation. Google's Confidential Computing VMs (with AMD SEV-SNP or Intel TDX) are available for Vertex AI workloads but not enabled by default. Azure Confidential Computing (with Intel SGX or AMD SEV-SNP) is available for custom deployments but not yet integrated into Foundry's managed services.

4.3 Compliance & Certification Matrix

Certification	AWS Bedrock	Google Vertex AI	Azure Foundry
SOC 1	Yes	Yes	Yes
SOC 2	Yes	Yes	Yes
SOC 3	Yes	Yes	Yes
ISO 27001	Yes	Yes	Yes
ISO 27017	Yes	Yes	Yes
ISO 27018	Yes	Yes	Yes
HIPAA	Eligible (BAA available)	Eligible (BAA available)	Eligible (BAA available)
PCI DSS	Yes	Yes	Yes
FedRAMP High	GovCloud only	Authorized (select services)	Azure Government
GDPR	Yes (EU regions)	Yes (EU regions)	Yes (EU regions, Data Zones)
EU AI Act	Readiness tools available	Readiness tools available	Readiness tools available
CSA STAR	Level 2	Level 2	Level 2

Certification	AWS Bedrock	Google Vertex AI	Azure Foundry
C5 (Germany)	Yes	Yes	Yes
IRAP (Australia)	Yes	Yes	Yes
ISMAP (Japan)	Yes	Yes	Yes

Chapter 5: Implementation Patterns — From Design to Production

5.1 Production Deployment Checklist

The following checklist captures the critical decisions and configurations required for a production cloud AI deployment. Each item represents a failure point observed in real-world implementations:

- **Model Selection:** Benchmark at least 3 models on your actual data. Use evaluation tools (Bedrock Evaluations, Gen AI Evaluation Service, Foundry evals). Don't select based on benchmarks alone—benchmark performance rarely correlates with domain-specific performance.
- **Prompt Engineering & Testing:** Version-control your system prompts. Test with adversarial inputs. Measure hallucination rate on a golden dataset. Implement A/B testing infrastructure before launch.
- **Guardrails Configuration:** Enable content filtering, PII detection, and grounding checks from day one. Set thresholds conservatively and loosen based on monitoring data. Document every threshold decision for compliance.
- **Network Isolation:** Configure PrivateLink / Private Service Connect / Private Link. Verify no public internet egress for inference traffic. Test connectivity from your VPC. Monitor for DNS resolution failures.
- **IAM / RBAC:** Apply least privilege. Separate roles for model deployment, agent creation, knowledge base management, and production invocation. Use service-linked roles for automation. Rotate API keys on schedule.
- **Observability:** Enable CloudTrail / Cloud Audit Logs / Azure Monitor. Configure model invocation logging for compliance-critical workloads. Set up dashboards for latency (P50/P95/P99), error rate, token usage, and cost. Create alerts for anomalies.
- **Cost Controls:** Set token budgets per project/team. Implement circuit breakers for runaway agents. Use batch inference for non-real-time workloads. Evaluate prompt caching for repeated context. Monitor cost per query and set alerts.
- **Disaster Recovery:** Deploy across multiple regions. Test failover scenarios. Maintain a fallback model (simpler, always-available) for degraded-mode operation. Document RTO/RPO for AI-dependent workflows.
- **Model Lifecycle:** Subscribe to deprecation notifications. Maintain an inventory of every model version in use. Schedule quarterly migration testing. Never use preview/experimental models in production.
- **Responsible AI:** Implement bias evaluation on your domain data. Test for disproportionate impact across demographic groups. Document model limitations in a model card. Establish a human review process for high-stakes decisions.

5.2 Anti-Patterns — What Not to Do

- **The God Agent:** A single agent with access to all tools, all data, and no scope boundaries. Failure mode: unpredictable behavior, impossible to debug, security nightmare. Fix: decompose into specialized agents with clear responsibilities and limited tool access.
- **RAG Without Evaluation:** Deploying a RAG pipeline and assuming it works because the demo looked good. Failure mode: silent quality degradation as data changes. Fix: implement retrieval evaluation with golden Q&A pairs; monitor recall and precision in production.

- **Frontier Model for Everything:** Using GPT-5 or Claude Opus for every request regardless of complexity. Failure mode: 10x cost with no quality improvement for simple tasks. Fix: intelligent routing, model distillation, or explicit model selection by task type.
- **Ignoring Model Deprecation:** Using model version aliases (“latest”) in production. Failure mode: behavior changes silently when the alias points to a new version; or model breaks when version is retired. Fix: pin specific model versions; test before upgrading.
- **No Guardrails in Development:** Disabling safety features during development to “move fast.” Failure mode: unsafe patterns are baked into the application design and are expensive to retrofit. Fix: guardrails from day one, even in dev environments.
- **Shared Identity for All Agents:** Running all agents under a single service account. Failure mode: lateral movement—if one agent is compromised, attacker gets access to all tools and data. Fix: one identity per agent with minimum required permissions.

5.3 Scaling Patterns

5.3.1 Horizontal Scaling

- **Bedrock:** Automatically scales. No action required for on-demand. For provisioned throughput, purchase additional model units. Cross-Region Inference distributes load geographically.
- **Vertex AI:** Provisioned Throughput via GSUs. Global endpoint for overflow traffic. GKE autoscaling for self-managed model serving.
- **Foundry:** Global Standard deployment auto-scales. Provisioned Throughput Units (PTUs) for guaranteed capacity. Bring-Your-Own-Runtime with AKS autoscaling for maximum control.

5.3.2 Cost-Performance Optimization

- **Model Distillation (Bedrock):** Distill a frontier model into a smaller, faster version tuned for your specific use case. AWS reports 500% faster inference and 75% lower cost. Ideal for high-volume, repetitive tasks (invoice extraction, ticket classification).
- **Prompt Caching (All Platforms):** Cache large system prompts and repeated context. Vertex AI supports 1-hour TTL for Anthropic Claude prompt caching. Bedrock caches across requests to the same model. Saves 50–90% on input tokens.
- **Batch Inference (Bedrock):** Submit datasets for asynchronous processing at up to 50% discount. Ideal for evaluation pipelines, bulk document processing, and content generation.
- **Spot/Preemptible Training (SageMaker/Vertex AI):** Up to 90% savings for training jobs. Requires checkpointing. SageMaker’s managed spot training handles interruption and restart automatically.

Chapter 6: Support, Maintenance & Operational Lifecycle

6.1 Model Lifecycle Management

Cloud AI models follow a lifecycle: Preview → GA → Deprecation → Retirement. Managing this lifecycle is a critical operational discipline:

Phase	AWS Bedrock	Google Vertex AI	Microsoft Foundry
Preview	Available for testing; not recommended for production; may change without notice	Public Preview; subject to Pre-GA terms; limited support	Preview; may not have SLA; available in limited regions
GA	Fully supported; covered by AWS SLA; model versioned	Stable model; covered by Gemini Online Inference SLA	GA; covered by Azure SLA; model versioned
Deprecation Notice	Announced when successor model is released	Retirement date announced when next stable version releases	Announced in Foundry blog and docs; typically 6–12 months notice
Access Block	1 month before retirement: new deployments blocked	1 month before retirement: new access blocked for online/batch/tuning	Model removed from catalog; existing deployments may continue temporarily
Retirement	Model no longer accessible or supported	Model no longer accessible or supported	Model no longer accessible
Migration Support	Decision guide docs; Bedrock Evaluations for comparison	Migration guide with step-by-step instructions; recommended upgrades listed	Foundry Models docs list recommended upgrades; SDK migration guides

⚠ Real-world impact example: Google deprecated gemini-2.0-flash-001 on March 6, 2026. New projects could no longer access it. The recommended upgrade path was gemini-2.5-flash. Organizations that hadn't tested 2.5 Flash experienced unexpected behavior changes in production. Lesson: always test successor models proactively, not when forced by deprecation.

6.2 SDK & API Versioning

All three clouds underwent major SDK consolidation in early 2026:

- Azure:** The azure-ai-projects 2.0.0 SDK (Python, JS/TS, Java, .NET) consolidates agents, inference, evaluations, and memory into a single AIProjectClient. The standalone azure-ai-agents package is eliminated. The v1 Azure ML SDK reaches end of support on June 30, 2026.
- AWS:** The Bedrock and SageMaker SDKs remain separate but are accessed from the same SageMaker Unified Studio. The boto3 bedrock-runtime and bedrock-agent-runtime clients are the primary interfaces. The Strands SDK (open-source) is the agent framework.
- Google:** The Vertex AI SDK is the primary interface, now transitioning to the Agent Platform namespace. The ADK (Agent Development Kit) is a separate open-source package for agent orchestration.

6.3 Monitoring & Alerting Strategy

A production cloud AI deployment requires monitoring at four levels:

- Infrastructure Level:** GPU/TPU utilization, memory pressure, network throughput, endpoint health. Use CloudWatch (AWS), Cloud Monitoring (GCP), Azure Monitor.

- **Service Level:** API latency (TTFT, TPS), error rates (4xx, 5xx), token consumption, cache hit rates, guardrail trigger rates. Create SLOs for P99 latency and error rate.
- **Application Level:** Agent completion rates, tool call success/failure, retrieval quality metrics (recall, precision, NDCG), user satisfaction (explicit feedback, implicit signals like retry rate).
- **Business Level:** Cost per query, cost per task completion, deflection rate (for customer service), accuracy on business-specific metrics, human escalation rate.

6.4 Incident Response for AI Systems

AI-specific incidents require extended playbooks beyond traditional application incidents:

- **Model Misbehavior:** Model generates harmful, biased, or nonsensical output. Response: enable maximum guardrail severity immediately; review recent prompt changes; check if model version changed; roll back to last known good configuration; escalate to provider if systematic.
- **Prompt Injection Attack:** Adversary manipulates model behavior through crafted inputs. Response: review audit logs for attack patterns; tighten input guardrails; add content filtering rules; consider input sanitization layer; report to provider for model-level mitigation.
- **Data Leakage via Agent:** Agent exposes sensitive data through tool calls or generated content. Response: immediately disable affected agent; review tool permissions; audit all agent actions in the incident window; tighten IAM and Cedar/Policy constraints; implement output filtering.
- **Cost Runaway:** Agent enters infinite loop, burning tokens. Response: circuit breaker activates (if configured); if not, manually disable agent; review token budgets and max_iterations settings; implement cost alerts and auto-shutdown thresholds.
- **Retrieval Poisoning:** Adversary plants misleading content in the knowledge base. Response: audit recent data source changes; review ingestion logs; implement data validation in the ingestion pipeline; add provenance tracking; consider adversarial testing of the retrieval pipeline.

Chapter 7: Trade-offs, Constraints & Decision Analysis

7.1 Fundamental Trade-offs in Cloud AI

Every architectural decision in cloud AI involves trade-offs. Understanding these enables informed decision-making rather than following vendor recommendations blindly:

7.1.1 Latency vs. Cost vs. Quality

This is the “iron triangle” of cloud AI. You can optimize for any two, but the third will suffer:

- **Low Latency + High Quality = High Cost:** Use frontier models (GPT-5, Claude Opus, Gemini Pro) with provisioned throughput. Fast and accurate, but expensive at scale.
- **Low Latency + Low Cost = Lower Quality:** Use small models (Phi, Ministral 3B, Nova Micro) on-demand. Fast and cheap, but limited reasoning capability.
- **High Quality + Low Cost = Higher Latency:** Use frontier models with batch inference or prompt caching. Accurate and affordable, but not real-time.

Optimal strategy: Intelligent routing. Use a classifier to send simple queries to small models (low latency, low cost) and complex queries to frontier models (high quality). Bedrock’s Intelligent Prompt Routing automates this. On other platforms, implement a routing layer in your application.

7.1.2 Managed vs. Self-Managed

Dimension	Fully Managed (Bedrock/Vertex AI/Foundry)	Self-Managed (SageMaker/GKE/AKS)
Control	Limited to platform configuration options	Full control over model, serving, and infrastructure
Operational Burden	Near-zero; provider handles everything	Significant; MLOps team required
Cost at Low Volume	Efficient (pay-per-token)	Expensive (minimum instance/GPU cost)
Cost at High Volume	Can be expensive (per-token at scale)	More efficient (amortized GPU cost)
Model Customization	Fine-tuning, distillation, RAG	Architecture changes, custom training, RLHF, quantization
Latency Predictability	Variable (shared infrastructure)	Consistent (dedicated resources)
Compliance Control	Provider-managed certifications	Full control, full responsibility
Vendor Lock-in	Moderate (unified APIs help, but orchestration is proprietary)	Low (standard ML frameworks)
Time to Production	Hours to days	Weeks to months

7.1.3 Single-Cloud vs. Multi-Cloud AI

- **Single-Cloud Advantages:** Unified identity and access management, consistent networking, single billing, deep service integration, lower operational complexity, better support from a single vendor.
- **Multi-Cloud Advantages:** Best-of-breed model access (Gemini native on GCP, GPT native on Azure), reduced vendor dependency, geographic flexibility, competitive pricing leverage.
- **Multi-Cloud Reality:** Most enterprises use 2+ clouds by 2026. The practical approach is to designate a primary AI platform aligned with your dominant cloud investment and use cross-

cloud APIs for specific model access. All platforms now offer models from multiple providers, reducing the need for multi-cloud purely for model diversity.

7.1.4 Open-Weight vs. Proprietary Models

Dimension	Proprietary (GPT, Claude, Gemini)	Open-Weight (Llama, Mistral, DeepSeek, Gemma)
Performance	Frontier reasoning, coding, analysis	Competitive for many tasks; gap closing rapidly
Customization	Fine-tuning within platform constraints	Full fine-tuning, quantization, architecture modification
Cost	Higher per-token; no self-hosting option	Lower per-token on managed; dramatically lower self-hosted
Data Privacy	Provider processes your data (within contractual limits)	Full control when self-hosted; managed serving still processed by provider
Transparency	Black box (no weight access)	Weights available; training data increasingly documented
Support	Provider SLA and support	Community support; commercial support from model vendor
Regulatory	Harder to audit for EU AI Act compliance	Easier to audit; full access to model internals
Availability	Provider-controlled; can be deprecated	Self-hosted = always available; managed = provider-controlled

7.2 Platform-Specific Constraints & Limitations (May 2026)

- **AWS Bedrock:** Hosted agents not yet available (managed harness is preview). No native A2A protocol. Knowledge Base ingestion is asynchronous (no real-time sync). Guardrails don't support custom ML models for content classification.
- **Google Vertex AI / Agent Platform:** ADK is preview (not GA). Provisioned Throughput excludes preview models. Partner models (Claude) available only in limited regions. Agent Engine billing can be unpredictable for long-running agents.
- **Azure Foundry:** Hosted agents limited to North Central US. Azure ML SDK v1 end of support June 2026 (forced migration). Memory in Agent Service is preview. Voice Live requires separate Foundry agent—no standalone mode.
- **Copilot Studio:** Limited to Microsoft-approved models. Cannot access Foundry's full model catalog. Workflow complexity limited compared to code-first approaches. Teams-only deployment for basic (non-standalone subscription) plans. Classic chatbots in Teams deprecated after June 2026.

Appendix: Reference Architecture Patterns

A.1 Enterprise RAG Architecture (AWS)

Components: S3 (document store) → Bedrock Knowledge Base (ingestion, chunking, embedding) → OpenSearch Serverless (vector store) → Bedrock Agent (orchestration) → Bedrock Guardrails (safety) → CloudFront/API Gateway (client access). **Security layer:** PrivateLink for all service-to-service communication; KMS CMK for encryption; CloudTrail for audit; IAM roles with least privilege. **Cost optimization:** Titan Embeddings V2 at 256 dimensions for cost-sensitive corpora; Nova Micro for simple queries via intelligent routing; batch inference for document processing.

A.2 Multi-Agent Customer Service (Azure)

Components: Azure Front Door (routing) → Foundry Agent Service (orchestration) → Triage Agent (intent classification, Phi model) → FAQ Agent (RAG via Foundry IQ + AI Search) → Escalation Agent (CRM integration via MCP to Salesforce) → Human Handoff (Teams integration via Copilot Studio). **Security layer:** Entra ID for user authentication; Private Link for all endpoints; Prisma AIRS for prompt injection detection; PII NextGen for data protection; Azure Policy for governance. **Monitoring:** Continuous evaluation sampling production traffic; Azure Monitor dashboards; alerts on quality regression and cost anomalies.

A.3 Data-Grounded Analytics Agent (Google Cloud)

Components: BigQuery (data warehouse) → Vertex AI Search (retrieval with hybrid search) → ADK Agent Network (orchestrator agent + SQL agent + visualization agent + report agent) → Agent Engine Runtime (managed compute) → Gemini Enterprise App (user interface). **Infrastructure:** TPU 8i for inference; Managed Lustre for model weight caching; Virgo Network for inter-chip communication. **Security layer:** VPC Service Controls perimeter; Cloud IAM agent identities; Model Armor for runtime protection; Cloud Audit Logs for compliance. **Data:** Knowledge Catalog for autonomous data tagging; BigQuery-native Gemma embeddings for vector search; text-to-SQL via Tools for Data Agents.

End of Deep Technical Guide — Version 3.0 — May 2026

This document reflects the state of cloud AI services as of May 2026. Services evolve rapidly—validate current capabilities against provider documentation before implementation.